

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
МЕХАНИКО-МАТЕМАТИЧЕСКИЙ ФАКУЛЬТЕТ

Кафедра дифференциальных уравнений и системного анализа

ОБУЧЕНИЕ С ПОДКРЕПЛЕНИЕМ ДЛЯ ИГРЫ «ЗМЕЙКА»

Курсовая работа

Полегина Александра Дмитриевича

студента 3-го курса
специальности 6-05-0533-08
«Компьютерная математика
и системный анализ»

Научный руководитель:
кандидат физ.-мат. наук,
доцент С. Г. Красовский

Минск, 2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
1 АНАЛИЗ ЗАДАЧИ И ПОДХОДОВ К ЕЁ РЕШЕНИЮ	4
1.1 Постановка задачи управления в игре «Змейка»	4
1.2 Особенности обучения с подкреплением для дискретной среды .	4
1.3 Сравнение возможных подходов к решению	5
1.4 Обоснование выбора табличного Q-learning	6
2 ПРОЕКТИРОВАНИЕ ИГРОВОЙ СРЕДЫ И ОБУЧАЮЩЕГО АГЕНТА	8
2.1 Требования к игровой среде	8
2.2 Формирование пространства состояний	9
2.3 Множество действий и проектирование функции награды	10
2.4 Архитектура программной системы	11
3 РЕАЛИЗАЦИЯ ПРОГРАММНОЙ СИСТЕМЫ	13
3.1 Реализация логики игры	13
3.2 Реализация RL-среды и Q-learning агента	13
3.3 Реализация интерфейса и режимов работы программы	14
3.4 Сохранение обученной политики и структура модулей	15
4 ЭКСПЕРИМЕНТАЛЬНОЕ ИССЛЕДОВАНИЕ И АНАЛИЗ РЕЗУЛЬТАТОВ	17
4.1 Методика вычислительного эксперимента	17
4.2 Результаты обучения при различном числе эпизодов	18
4.3 Интерпретация результатов обучения	19
4.4 Анализ влияния гиперпараметров на обучение	19
4.5 Ограничения признакового описания состояния	20
4.6 Переход к более сложным методам: почему интересен DQN . . .	21
4.7 Ограничения метода и направления развития	22
ЗАКЛЮЧЕНИЕ	24
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ	25
ПРИЛОЖЕНИЕ А	26

ВВЕДЕНИЕ

Игра «Змейка» хорошо подходит для изучения обучения с подкреплением. В ней каждое действие сразу влияет на дальнейшую ситуацию на поле и на итоговый счёт. Поэтому на такой задаче удобно разбирать, как задаются состояния, действия и награды, и как агент постепенно учится принимать более удачные решения.

Тема работы актуальна потому, что обучение с подкреплением позволяет строить агентов, которые учатся на собственном опыте, а не по заранее прописанным правилам для каждой ситуации. Даже на сравнительно простой игре можно проследить все основные этапы разработки такой системы: создание среды, выбор признаков состояния, настройку награды, обучение и проверку результатов.

Цель курсовой работы — разработать игру «Змейка» на языке Python, реализовать алгоритм обучения с подкреплением для управления агентом и оценить, насколько хорошо он обучается.

Для достижения этой цели были поставлены следующие задачи:

- изучить основные идеи обучения с подкреплением и алгоритма Q-learning;
- разобрать требования к игровой среде и реализовать логику игры «Змейка»;
- задать пространство состояний, множество действий и функцию награды;
- реализовать агента и процесс накопления Q-таблицы;
- провести вычислительные эксперименты и оценить, как меняется качество игры во время обучения;
- подготовить таблицы, графики и изображения, которые объясняют устройство программы и поведение агента.

Объект исследования — программная игра «Змейка». Предмет исследования — методы обучения с подкреплением для выбора действий в дискретной игровой среде. В работе используются методы объектно-ориентированного программирования, имитационного моделирования и вычислительного эксперимента.

ГЛАВА 1

АНАЛИЗ ЗАДАЧИ И ПОДХОДОВ К ЕЁ РЕШЕНИЮ

1.1 Постановка задачи управления в игре «Змейка»

В этой задаче нужно построить агента, который управляет змейкой на дискретном поле и старается набрать как можно больше очков. На каждом шаге агент выбирает действие по текущему состоянию игры, после чего среда возвращает новое состояние и награду. Поэтому задачу удобно рассматривать как последовательное принятие решений.

Особенность игры в том, что хороший ход не всегда даёт пользу сразу. Действие может выглядеть безопасным сейчас, но через несколько шагов привести либо к пище, либо к столкновению. Из-за этого агенту нужно учитывать не только ближайшую ситуацию, но и накопленный опыт.

1.2 Особенности обучения с подкреплением для дискретной среды

Обучение с подкреплением подходит для задач, где агент сам ищет удачную стратегию, взаимодействуя со средой и получая награды за свои действия. В отличие от обучения с учителем, здесь нет набора правильных ответов. Агент учится на том, к чему приводят его решения.

На шаге t агент видит состояние s_t , выбирает действие a_t , получает награду r_t и переходит в состояние s_{t+1} . Для игры «Змейка» такой подход удобен, потому что:

- игра имеет дискретную структуру;
- число действий невелико;
- результат легко оценивать по счёту;
- поведение агента можно наглядно показать на экране.

Поэтому «Змейка» хорошо подходит для демонстрации базовых идей reinforcement learning на реальном проекте.

1.3 Сравнение возможных подходов к решению

Управлять змейкой можно по-разному. Самый простой вариант — набор правил и эвристик. В этом случае разработчик заранее прописывает, как агент должен реагировать на опасность и как двигаться к пище. Такой подход легко начать, но он плохо переносится на более сложные игровые ситуации.

Другой вариант — использовать алгоритмы поиска пути. Они могут строить маршрут к цели, но требуют отдельного планирования и хуже показывают сам процесс обучения на опыте.

Третий вариант — методы обучения с подкреплением, например табличный Q-learning или более сложные нейросетевые методы, такие как DQN. В нашем проекте состояние уже сведено к компактному набору признаков, поэтому можно применять табличный метод без лишнего усложнения.

Таблица 1.1 Сравнение подходов к управлению змейкой

Подход	Преимущества	Ограничения
Правила и эвристики	Простая реализация, предсказуемое поведение	Сложно покрыть все игровые ситуации, слабая адаптивность
Поиск траектории	Явный контроль маршрута, хорош для детерминированных сценариев	Требует дополнительного планирования на каждом шаге, хуже отражает накопление опыта
Табличный Q-learning	Интерпретируемость, простота обновления, естественная связь с дискретной средой	Ограниченная масштабируемость при росте пространства состояний
DQN и другие глубокие методы	Хорошо работают на более сложных наблюдениях и больших пространствах состояний	Требуют больше данных, вычислений и настройки

1.4 Обоснование выбора табличного Q-learning

В этой работе выбран табличный Q-learning. На это есть две основные причины. Во-первых, он хорошо подходит для учебного проекта, потому что его легко объяснять и анализировать. Во-вторых, в проекте используется вектор состояния из 11 бинарных признаков, а значит Q-значения ещё можно хранить в таблице.

Такой выбор даёт хороший баланс между простотой и пользой. С одной стороны, метод не перегружает проект лишней сложностью. С другой стороны, он позволяет получить реальный обучаемый агент и показать, как

меняется его поведение по мере накопления опыта.

ГЛАВА 2

ПРОЕКТИРОВАНИЕ ИГРОВОЙ СРЕДЫ И ОБУЧАЮЩЕГО АГЕНТА

2.1 Требования к игровой среде

Среда должна решать сразу две задачи. С одной стороны, она должна правильно реализовывать саму игру: движение змейки, появление пищи, рост длины и завершение партии при столкновении. С другой стороны, она должна быть удобной для обучения агента: возвращать состояние, принимать действие и выдавать численную награду.

В проекте используется поле размером 20×20 клеток. Змейка начинает движение из центра поля и сначала направлена вправо. Такое начальное положение удобно, потому что оно симметрично и не даёт преимущества какой-то одной стороне поля.

Таблица 2.1 Основные параметры игровой среды и обучения

Параметр	Значение	Назначение
Размер поля	20×20	Количество клеток по горизонтали и вертикали
Размер клетки	24 px	Масштаб визуализации в окне Pygame
FOOD_REWARD	10.0	Награда за поедание пищи
COLLISION_PENALTY	-10.0	Штраф за столкновение
STEP_PENALTY	-0.1	Небольшой штраф за каждый ход
ALPHA	0.1	Скорость обновления Q-оценок
GAMMA	0.9	Учёт будущих вознаграждений
EPSILON	1.0	Начальная интенсивность исследования
EPSILON_DECAY	0.995	Множитель снижения ϵ
EPSILON_MIN	0.05	Нижняя граница ϵ
TRAIN_EPISODES	300	Базовое число эпизодов обучения

2.2 Формирование пространства состояний

Одним из главных решений в проекте стал выбор состояния агента. Если хранить все координаты змейки и еды, число состояний быстро станет слишком большим. Поэтому в работе используется более компактный вариант — вектор из 11 бинарных признаков.

В этот вектор входят признаки опасности при движении прямо, влево и вправо, текущее направление змейки и положение пищи относительно головы. Такой набор даёт агенту ту информацию, которая действительно нужна для выбора следующего хода, и при этом не делает модель слишком сложной.

Такой подход важен не только для удобства реализации, но и для самой логики обучения. Агенту не обязательно знать полную геометрию поля в каждый момент времени, если для принятия ближайшего решения ему достаточно понимать три вещи: есть ли опасность рядом, куда он сейчас движется и с какой стороны находится цель. За счёт этого состояние получается компактным и хорошо подходит для табличного Q-learning.

Ещё одно достоинство такого описания состоит в том, что оно делает поведение агента более обобщаемым. Разные игровые ситуации, которые с точки зрения координат выглядят по-разному, могут совпадать по смыслу: например, во всех них еда находится справа, а впереди есть опасность. В табличном методе это особенно полезно, потому что позволяет использовать один и тот же набор оценок для типовых игровых шаблонов.

Таблица 2.2 Состав вектора состояния среды

№	Признак	Смысл
1	danger_straight	Столкновение при движении прямо
2	danger_left	Столкновение при повороте влево
3	danger_right	Столкновение при повороте вправо
4	dir_up	Текущая ориентация вверх
5	dir_down	Текущая ориентация вниз
6	dir_left	Текущая ориентация влево
7	dir_right	Текущая ориентация вправо
8	food_left	Пища расположена левее головы
9	food_right	Пища расположена правее головы
10	food_up	Пища расположена выше головы
11	food_down	Пища расположена ниже головы

2.3 Множество действий и проектирование функции награды

Вместо абсолютных направлений в проекте используются относительные действия: идти прямо, повернуть влево и повернуть вправо. Это упрощает обучение, потому что агент выбирает действие относительно своего текущего движения, а не относительно экрана.

Награда построена так, чтобы агенту было выгодно искать пищу и избегать бесполезных ходов. За поедание пищи он получает 10.0, за столкновение — штраф -10.0, а за обычный шаг — небольшой штраф -0.1. За счёт этого агент учится не просто долго жить, а двигаться к цели.

Выбор относительных действий особенно важен с точки зрения обучения. Если бы агент использовал абсолютные команды вроде «вверх», «вниз», «влево» и «вправо», ему пришлось бы отдельно учиться похожим ситуациям для разных текущих направлений движения. Относительная схема уменьшает эту избыточность: действие «повернуть влево» имеет один и тот же смысл независимо от того, куда змейка была направлена до этого. Это делает пространство действий меньше и помогает агенту быстрее накапливать полезный опыт.

Функция награды тоже играет ключевую роль. Положительная награда

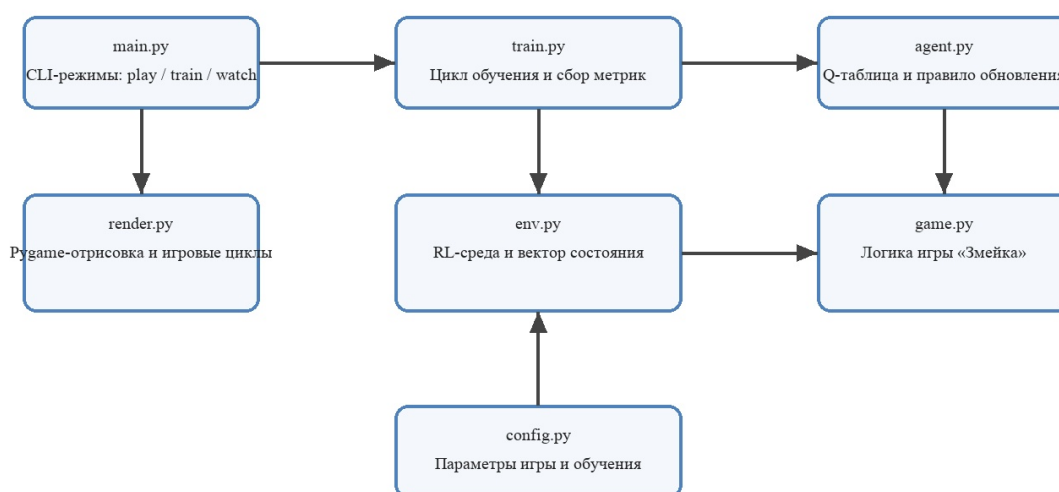
за пищу даёт агенту явную цель. Штраф за столкновение показывает, какие ситуации являются критически плохими. Небольшой штраф за обычный шаг нужен для того, чтобы агент не выбирал бесконечное безопасное движение по кругу, которое не приводит к росту счёта. Иначе в задаче могла бы появиться стратегия, при которой агент долго избегает смерти, но почти не решает основную задачу.

С точки зрения проектирования это означает, что награда задаёт само понятие «хорошего поведения». В данной работе хорошей считается стратегия, которая одновременно избегает столкновений, движется к пище и не тратит слишком много лишних шагов. Именно такой баланс и был заложен в систему вознаграждений.

2.4 Архитектура программной системы

Программа разбита на несколько модулей. Файл `game.py` отвечает за игровую механику, `env.py` превращает игру в среду для обучения, `agent.py` содержит Q-learning агента, `train.py` управляет обучением, `render.py` отвечает за визуализацию, а `main.py` запускает нужный режим работы.

Архитектура проекта Snake RL



Командный интерфейс запускает ручную игру, обучение или просмотр обученного агента. Во время обучения агент взаимодействует со средой, среда с

Рисунок 2.1 Структурная схема взаимодействия модулей проекта

Такое разбиение делает проект более понятным и удобным для дора-

ботки. Игровую логику, обучение и интерфейс можно рассматривать как отдельные части системы.

Кроме того, такая архитектура удобна и для анализа результатов. Если агент обучается плохо, можно отдельно проверять, правильно ли работает игровая логика, корректно ли формируется состояние, верно ли считается награда и не ошибается ли модуль выбора действия. Это особенно важно в учебном проекте, где цель состоит не только в получении работающей программы, но и в понимании того, как взаимодействуют все её части.

ГЛАВА 3

РЕАЛИЗАЦИЯ ПРОГРАММНОЙ СИСТЕМЫ

3.1 Реализация логики игры

Базовый класс `SnakeGame` отвечает за основную механику игры. В нём реализованы положение головы и тела змейки, проверка столкновений, рост длины после поедания пищи и выбор свободной клетки для новой еды.

Также в игре есть защита от мгновенного разворота на 180 градусов. Это соответствует обычным правилам «Змейки» и убирает некорректные переходы между состояниями.

Такое разделение полезно тем, что игровая логика остаётся независимой от обучения. Игра умеет корректно работать сама по себе: обновлять положение объектов, проверять конец партии и поддерживать состояние поля. Благодаря этому один и тот же игровой движок можно использовать и для ручного режима, и для обучения, и для наблюдения за уже обученным агентом.

3.2 Реализация RL-среды и Q-learning агента

Среда переводит игровую механику в форму, удобную для обучения. По текущему положению змейки строится вектор состояния, затем принимается одно из трёх действий, после чего среда возвращает награду, новое состояние и признак завершения эпизода.

Класс `QLearningAgent` хранит Q-таблицу в виде словаря. Ключом служит кортеж признаков состояния, а значением — список оценок для трёх действий. Если состояние встречается впервые, для него создаётся набор нулевых оценок.

Выбор действия строится по ϵ -жадному правилу: иногда агент делает случайный ход, а иногда выбирает лучший из уже известных. После каждого шага вызывается функция `update`, которая обновляет Q-значение для пары «состояние — действие».

Если рассмотреть один эпизод обучения по шагам, то процесс выглядит так. Сначала среда возвращает текущее состояние. Затем агент по этому

состоянию выбирает действие. После выполнения действия игра переходит в новую конфигурацию, а среда вычисляет награду. Далее агент получает новое состояние и обновляет соответствующее значение в Q-таблице. Такой цикл повторяется до тех пор, пока змейка не столкнётся с препятствием или не завершится текущая партия.

Преимущество такой схемы в том, что она хорошо согласуется с формальной моделью reinforcement learning. Каждый шаг обучения можно описать как переход из одного состояния в другое с фиксированным действием и численной наградой. Это делает программу не просто игровой реализацией, а полноценной учебной RL-средой.

3.3 Реализация интерфейса и режимов работы программы

Визуальная часть сделана на библиотеке `pygame`. Программа рисует поле, еду, змейку, строку состояния и окно завершения игры. Также есть ручной режим и режим наблюдения за уже обученным агентом [`pygame_docs`, 1].

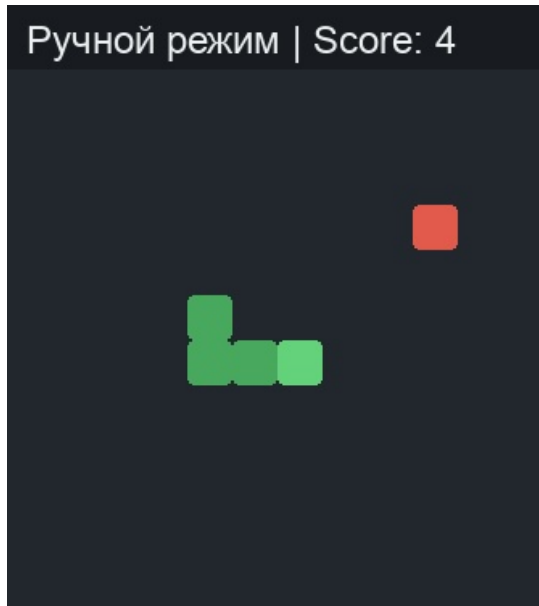


Рисунок 3.1 Интерфейс программы в режиме ручной игры

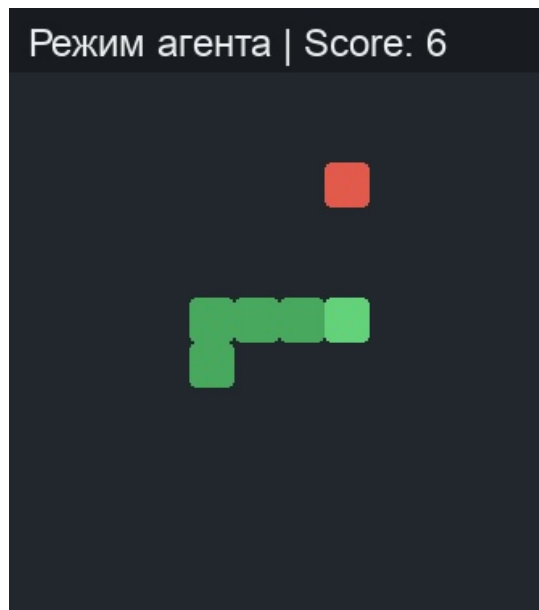


Рисунок 3.2 Интерфейс программы в режиме наблюдения за агентом

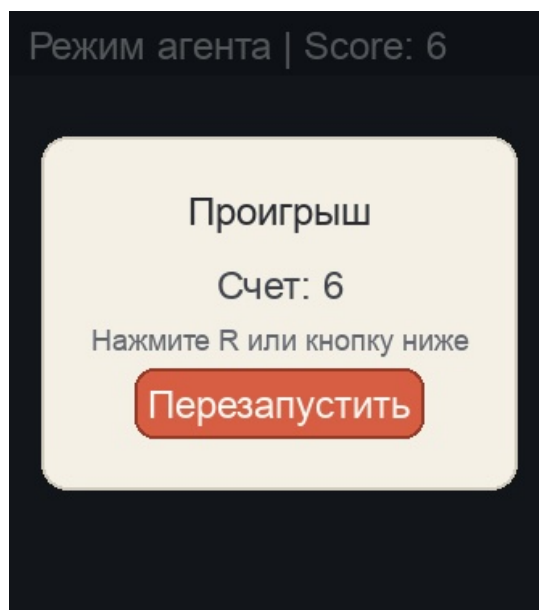


Рисунок 3.3 Модальное окно завершения партии и перезапуска

3.4 Сохранение обученной политики и структура модулей

После обучения таблица сохраняется в файл `artifacts/q_table.pkl`. Благодаря этому можно запускать агента в режиме `watch` без повторного обучения. Это удобно и для экспериментов, и для демонстрации работы программы.

Практическая ценность сохранения Q-таблицы состоит в том, что обучение и демонстрация результатов становятся независимыми друг от друга.

Не нужно каждый раз заново запускать длинную процедуру обучения, чтобы показать поведение агента. Можно один раз получить таблицу, а затем использовать её для серии визуальных запусков, сравнения поведения в разных ситуациях и подготовки иллюстраций для отчёта.

С точки зрения структуры проекта это решение также полезно тем, что отделяет этап накопления опыта от этапа использования уже готовой стратегии. Такая схема особенно удобна для курсовой работы: она позволяет сначала описать процесс обучения как вычислительный эксперимент, а затем отдельно показать практический результат в виде наблюдаемого игрового поведения.

Таблица 3.1 Назначение основных модулей проекта

Файл	Назначение
<code>main.py</code>	Разбор аргументов командной строки и переключение режимов <code>play/train/watch</code>
<code>train.py</code>	Создание среды и агента, организация эпизодов обучения, вычисление итоговых метрик
<code>agent.py</code>	Хранение Q-таблицы, ϵ -жадный выбор действий, обновление оценок, сохранение и загрузка
<code>env.py</code>	Преобразование игры в RL-среду с вектором состояния и системой наград
<code>game.py</code>	Низкоуровневая логика игры, проверка столкновений и генерация пищи
<code>render.py</code>	Визуализация, ручное управление, режим наблюдения за агентом
<code>tests/</code>	Набор автоматических тестов для проверки основных компонентов

Каждый из этих модулей выполняет самостоятельную роль, но максимальный эффект они дают именно в связке. Игровой модуль формирует динамику среды, среда переводит её в RL-формат, агент учится на переходах, модуль обучения организует серию эпизодов, а визуализация позволяет наглядно увидеть итог поведения. Благодаря этому проект выглядит как завершенная система, а не как набор разрозненных программных фрагментов.

ГЛАВА 4

ЭКСПЕРИМЕНТАЛЬНОЕ ИССЛЕДОВАНИЕ И АНАЛИЗ РЕЗУЛЬТАТОВ

4.1 Методика вычислительного эксперимента

Чтобы оценить качество обучения более надёжно, была проведена серия экспериментов с разным числом эпизодов. Для каждого значения числа эпизодов агент обучался 5 раз с независимой случайной инициализацией. После этого каждая полученная Q-таблица оценивалась на 100 тестовых играх без исследования, то есть при выборе действий только по уже выученным оценкам. В качестве основного показателя использовался средний score на тестовых играх, а в качестве дополнительного показателя — средний лучший score, достигнутый в серии оценивания.

Таблица 4.1 Схема вычислительного эксперимента

Характеристика	Описание
Число независимых обучений	5 запусков для каждого набора эпизодов
Число тестовых игр	100 игр для каждой обученной Q-таблицы
Число эпизодов обучения	50, 100, 200, 300, 500, 800, 1000, 1200
Основной показатель	Средний score на тестовых играх
Дополнительный показатель	Средний лучший score в серии тестов
Назначение эксперимента	Оценка зависимости качества игры от объёма обучающего опыта

4.2 Результаты обучения при различном числе эпизодов

Таблица 4.2 Результаты обучения агента

Эпизоды	Средний score	Средний лучший score
50	4.262	15.800
100	7.544	19.200
200	9.760	19.400
300	9.912	18.800
500	13.958	21.400
800	15.422	21.200
1000	15.494	21.400
1200	16.230	21.600

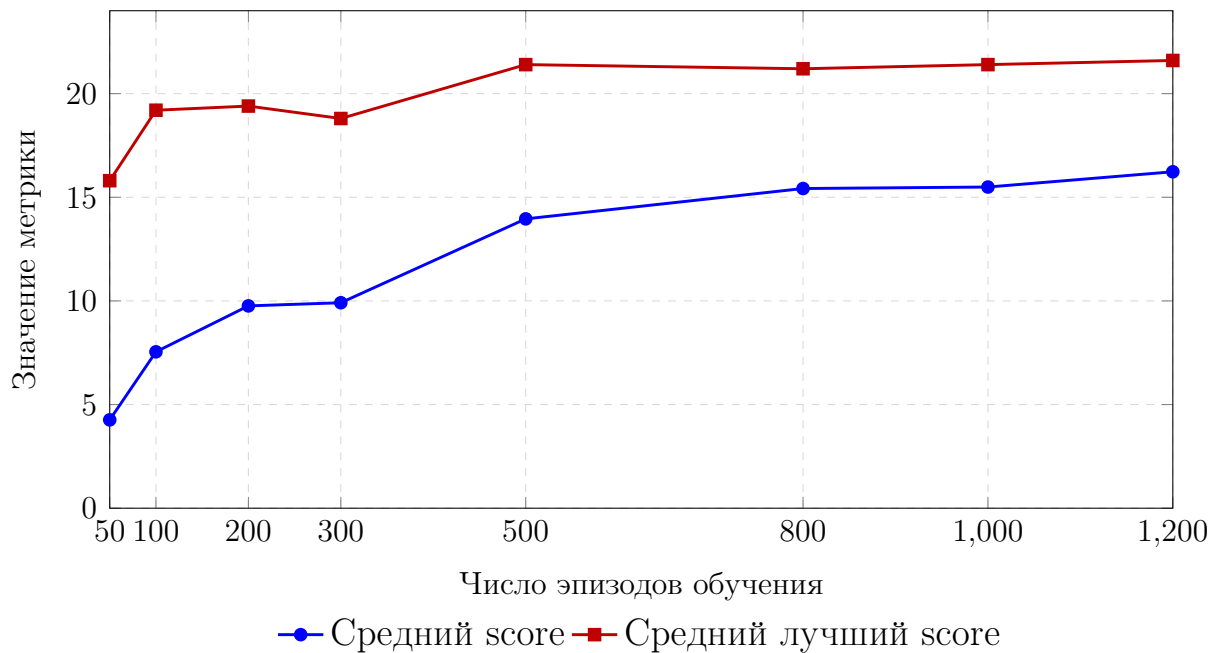


Рисунок 4.1 Зависимость среднего и лучшего результата от числа эпизодов обучения

График и таблица показывают устойчивый рост качества игры по мере накопления опыта. Наиболее заметное улучшение наблюдается в диапазоне от 50 до 500 эпизодов: средний score быстро увеличивается, а лучший результат выходит на уровень около 20 очков. После 800 эпизодов рост замедляется, что указывает на постепенный выход обучения на плато. Это означает, что

агент действительно усваивает полезную стратегию, но дальнейшее увеличение числа эпизодов даёт уже не столь большой прирост качества.

4.3 Интерпретация результатов обучения

Для машинного обучения важен не только рост метрик, но и то, почему он происходит. В этом проекте агент постепенно заполняет Q-таблицу полезными оценками действий в часто встречающихся состояниях. Из-за этого на поздних этапах он реже выбирает опасные ходы и чаще движется так, чтобы сохранить безопасность или добраться до пищи.

Нужно учитывать, что обучение остаётся случайным процессом. Разные запуски могут давать немного разные результаты из-за случайного положения пищи и исследовательских действий агента. Но общий рост показателей всё равно хорошо заметен.

4.4 Анализ влияния гиперпараметров на обучение

Результат работы агента зависит не только от самого алгоритма, но и от выбора гиперпараметров. В данном проекте особенно важны коэффициент обучения α , коэффициент дисконтирования γ , параметр исследования ε и скорость его уменьшения. Даже при одной и той же игровой среде изменение этих величин может заметно повлиять на скорость обучения и на устойчивость итоговой стратегии.

Коэффициент α определяет, насколько сильно новые наблюдения влияют на уже накопленные оценки. Если сделать α слишком маленьким, агент будет учиться очень медленно: новые удачные и неудачные ситуации почти не будут менять таблицу. Если же α окажется слишком большим, оценки начнут резко колебаться, и обучение станет менее устойчивым. В текущей работе используется значение 0.1, которое даёт достаточно плавное, но заметное обновление знаний.

Коэффициент γ показывает, насколько агент учитывает будущую награду. При слишком маленьком γ агент будет ориентироваться почти только на ближайший шаг и хуже понимать, что безопасный манёвр сейчас может привести к пище через несколько ходов. При слишком большом значении обучения может становиться более чувствительным к далёким и неопределённым

последствиям. Значение 0.9 в этой работе позволяет учитывать долгосрочную цель, но при этом не делает поведение чрезмерно инерционным.

Параметр ϵ отвечает за баланс между исследованием и использованием уже найденных удачных действий. На старте обучения агенту важно часто пробовать разные ходы, поэтому начальное значение ϵ равно 1.0. Затем этот параметр постепенно уменьшается. Если уменьшать его слишком быстро, агент может слишком рано закрепиться на слабой стратегии. Если уменьшать слишком медленно, обучение будет дольше оставаться случайным. Поэтому множитель $\text{EPSILON_DECAY} = 0.995$ и нижняя граница $\text{EPSILON_MIN} = 0.05$ здесь выбраны как разумный компромисс между поиском новых действий и закреплением уже найденных.

Таблица 4.3 Роль гиперпараметров обучения

Параметр	Роль	Слишком малое значение	Слишком большое значение
α	Скорость обновления Q-оценок	Обучение идёт медленно, агент слабо реагирует на новый опыт	Оценки сильно колеблются, обучение становится менее устойчивым
γ	Учёт будущей награды	Агент ориентируется почти только на ближайший шаг	Поведение может стать слишком зависимым от далёких последствий
ϵ	Доля случайных действий	Агент мало исследует среду и может рано застрять в слабой стратегии	Агент слишком долго действует почти случайно
EPSILON_DECAY	Скорость уменьшения исследования	Исследование заканчивается слишком быстро	Исследование сохраняется слишком долго

4.5 Ограничения признакового описания состояния

Сильной стороной проекта является компактное описание состояния, но именно оно же задаёт и границы применимости метода. Вектор из 11 би-

нарных признаков хорошо работает в учебной постановке, потому что хранит только самую важную информацию: опасность рядом, текущее направление движения и относительное положение пищи. Этого достаточно, чтобы агент научился избегать простых столкновений и выбирать более разумные ходы.

Однако такое описание не передаёт полную картину игрового поля. Агент не знает точную форму собственного тела на дальних клетках, не видит длинные обходные маршруты и не получает детальной информации о будущих ловушках, которые могут возникнуть через несколько ходов. Поэтому при усложнении среды или при увеличении длины змейки такая модель может начать терять качество.

С практической точки зрения это важное наблюдение для курсовой работы. Оно показывает, что успех агента определяется не только формулой Q-learning, но и тем, насколько удачно подобраны признаки состояния. Если признаки слишком бедные, агент просто не сможет отличить по-настоящему хорошие ситуации от плохих. Если же признаки сделать слишком подробными, пространство состояний вырастет настолько, что табличный метод станет неудобным.

Таким образом, текущее представление состояния является компромиссным. Оно достаточно простое для табличного метода и достаточно информативное для учебной игры, но уже на этом уровне показывает, где начинаются ограничения такого подхода.

4.6 Переход к более сложным методам: почему интересен DQN

Когда пространство состояний становится слишком большим, табличный Q-learning начинает терять эффективность. Для каждой новой комбинации признаков нужно хранить отдельные значения, а число таких комбинаций быстро растёт. В более сложных задачах, где агент должен учитывать полную картину поля, такой способ хранения уже плохо масштабируется.

Именно поэтому логичным следующим шагом выглядит переход к DQN и другим методам глубинного обучения с подкреплением. В этом случае Q-функция хранится не в таблице, а аппроксимируется нейронной сетью. Это позволяет работать с более богатыми наблюдениями и переносить знания между похожими состояниями, а не учить каждую ситуацию почти отдельно.

Для данной курсовой работы переход к DQN не был основной целью, поскольку сама задача строилась как понятная и интерпретируемая учебная реализация. Тем не менее сравнение с DQN важно с методической точки зрения. Оно показывает, что выбранный в проекте табличный подход хорошо подходит для простых и компактных постановок, но при дальнейшем усложнении задачи более естественным становится использование нейросетевых моделей.

Таблица 4.4 Сравнение табличного Q-learning и DQN

Критерий	Табличный Q-learning	DQN
Представление Q-функции	Явная таблица значений	Нейронная сеть
Подходящие состояния	Компактные и дискретные	Более сложные и высокоразмерные
Интерпретируемость	Высокая, проще объяснять шаги обучения	Ниже, так как поведение задаётся параметрами сети
Требования к ресурсам	Небольшие	Выше по памяти, времени и настройке
Уместность для текущей работы	Очень высокая	Полезен как направление дальнейшего развития

4.7 Ограничения метода и направления развития

Главное ограничение текущего решения связано с тем, что здесь используется табличный Q-learning. Он хорошо работает, пока состояние описывается компактным набором признаков. Но если перейти к более сложным наблюдениям, например к полному изображению поля, число состояний станет слишком большим.

В дальнейшем можно подробнее изучить влияние параметров α , γ и ε , попробовать другое описание состояния, увеличить размер поля и провести дополнительные серии экспериментов с разными схемами награды. Это позволило бы лучше понять, какие именно проектные решения сильнее всего влияют на поведение агента.

Кроме того, перспективным направлением остаётся переход к более

сложным методам, например DQN. Такой шаг был бы особенно полезен в тех задачах, где табличный метод уже не справляется из-за большого числа состояний или из-за необходимости работать с более подробным описанием игрового поля.

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы была создана программная система игры «Змейка» и реализован агент, обучающийся методом Q-learning. Рассмотрены основы обучения с подкреплением, спроектирована игровая среда, выбрано представление состояния и система наград.

Практическая часть показала, что даже табличный Q-learning позволяет агенту постепенно улучшать стратегию. В ходе вычислительных экспериментов средний результат после 100 эпизодов составил 7.544 очка, после 500 — 13.958 очка, а после 1200 — 16.230 очка. Средний лучший результат серии вырос с 15.800 до 21.600 очка. Это подтверждает, что по мере обучения агент принимает более удачные решения, хотя на поздних этапах наблюдается замедление роста качества.

Можно сделать вывод, что поставленная цель достигнута: игра реализована, обучаемый агент создан и его работа проанализирована. Полученные результаты показывают, что выбранное представление состояния, относительные действия и система наград позволяют формировать эффективную стратегию даже без использования нейронных сетей.

Состояние агента задано в виде вектора из 11 бинарных признаков, содержащих информацию о ближайших опасностях, текущем направлении движения и положении пищи. Это позволяет сохранить компактность пространства состояний, что особенно важно для табличного метода Q-learning.

Существенную роль играет система наград: положительное вознаграждение за поедание пищи, штраф за столкновение и небольшой штраф за каждый шаг формируют баланс между выживанием и поиском еды. Также важен баланс между исследованием среды и использованием уже изученных действий: сначала агент активно исследует, а затем закрепляет более успешную стратегию.

Дальнейшее развитие работы может включать исследование влияния параметров α , γ и ϵ , изменение способа представления состояния, увеличение размера игрового поля или переход к более сложным методам обучения, таким как DQN, которые лучше масштабируются на больших пространствах состояний.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

- [1] Eric Matthes. *Изучаем Python: программирование игр, визуализация данных, веб-приложения*. 3-е изд. 2023.
- [2] Ричард Саттон и Эндрю Барто. *Обучение с подкреплением. Введение*. Москва: Диалектика, 2020.

ПРИЛОЖЕНИЕ А

Исходный код проекта на GitHub

